

WEEK-8

Task-1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
GNU nano 8.7.1                                memory.c *

printf("malloc: ");
for (int i = 0; i < 5; i++)
    printf("%d ", a[i]);
printf("\n");

// calloc
b = calloc(5, sizeof(int));

if (b == NULL) {
    printf("calloc failed\n");
    free(a);
    return 1;
}

printf("calloc: ");
for (int i = 0; i < 5; i++)
    printf("%d ", b[i]);
printf("\n");

// realloc
temp = realloc(a, 10 * sizeof(int));

if (temp == NULL) {
    printf("realloc failed\n");
    free(a);
    free(b);
    return 1;
}

a = temp;

for (int i = 5; i < 10; i++)
    a[i] = i + 1;

printf("After realloc: ");
for (int i = 0; i < 10; i++)
    printf("%d ", a[i]);
printf("\n");

// Free memory
free(a);
free(b);

printf("Memory freed successfully.\n");

return 0;
}
```

Output:

```
[1]~$ ./maps
pavani@DESKTOP-380LEIN:~/practical$ nano memory.c
pavani@DESKTOP-380LEIN:~/practical$ gcc -g memory.c -o memory
pavani@DESKTOP-380LEIN:~/practical$ ./memory
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
Memory freed successfully.

pavani@DESKTOP-380LEIN:~/practical$ valgrind --leak-check=full ./memory
==2465== Memcheck, a memory error detector
==2465== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2465== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2465== Command: ./memory
==2465==
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
Memory freed successfully.
==2465==
==2465== HEAP SUMMARY:
==2465==   in use at exit: 0 bytes in 0 blocks
==2465==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==2465==
==2465== All heap blocks were freed -- no leaks are possible
==2465==
==2465== For lists of detected and suppressed errors, rerun with: -s
==2465== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
pavani@DESKTOP-380LEIN:~/practical$ gcc -g memory.c -o memory
pavani@DESKTOP-380LEIN:~/practical$ valgrind --leak-check=full ./memory
==2494== Memcheck, a memory error detector
==2494== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2494== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2494== Command: ./memory
==2494==
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
Memory freed successfully.
==2494==
==2494== HEAP SUMMARY:
==2494==   in use at exit: 0 bytes in 0 blocks
==2494==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==2494==
==2494== All heap blocks were freed -- no leaks are possible
==2494==
==2494== For lists of detected and suppressed errors, rerun with: -s
==2494== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Report:

The program successfully demonstrated dynamic memory allocation using malloc(), calloc(), realloc(), and free().

Different allocation methods were tested and their behavior was observed.

Valgrind was used to monitor memory usage and identify memory leaks.

The program showed no memory leaks when all allocated memory was properly released.

Task-2 : Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Code:

```
GNU nano 8.7.1 COW.C *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int *data = malloc(100000 * sizeof(int));

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 100000; i++)
        data[i] = i;

    printf("Before fork:\n");
    printf("Parent PID: %d\n", getpid());
    printf("Data address: %p\n", (void *)data);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        printf("\nChild process:\n");
        printf("Child PID: %d\n", getpid());
        printf("Data address: %p\n", (void *)data);

        printf("Modifying data in child...\n");

        data[0] = 999;

        printf("Child data[0] = %d\n", data[0]);

        sleep(3);
    }
    else {
```

Output:

```
==2494== ERROR SUMMARY: 0 errors from 0 contexts
pavani@DESKTOP-380LEIN:~/practical$ nano cow.c
pavani@DESKTOP-380LEIN:~/practical$ gcc cow.c -o cow
pavani@DESKTOP-380LEIN:~/practical$ ./cow
Before fork:
Parent PID: 2539
Data address: 0x745c4fb9e010

Child process:
Child PID: 2540
Data address: 0x745c4fb9e010
Modifying data in child...
Child data[0] = 999

Parent process:
Parent data[0] = 0
pavani@DESKTOP-380LEIN:~/practical$
```

Report: The Copy-on-Write behavior of fork() was successfully demonstrated. After fork(), the parent and child initially share the same physical memory pages. When the child modified the data, Linux created a private copy of the affected page. The parent retained the original value while the child received the modified value, demonstrating Copy-on-Write.